

LightNobel and FPGA Triangle-Multiplication Kernels: Technical Summary

This document summarizes (1) the operation of LightNobel (Han, Choi, Kim, ISCA 2025, arXiv 2505.05893), a hardware-software co-designed accelerator for protein structure prediction models (PPMs); (2) our FPGA implementation of the Triangle Multiplication einsum on an AMD Alveo U55C, versions v0-v3; and (3) the analytical methods used to evaluate them. Statements marked (*our interpretation*) are not stated explicitly in the paper.

1. Background: ESMFold and the Pair Representation

1.1 Pipeline

ESMFold, the baseline model used by LightNobel, predicts a 3D structure from an amino-acid sequence of length L in three stages:

Stage	Function	Main data
Input embedding	ESM-2 (3B) protein language model	sequence representation $s : L \times 1024$
Folding trunk	48 Protein Folding Blocks, up to 4 recycling iterations	s and pair representation $z : L \times L \times 128$
Structure module	predicts atom coordinates; distogram head from z	3D structure

Each element $z[i][j]$ (a *token* of the pair representation) is a 128-channel vector describing the relation between residues i and j . Every block updates the state residually ($z \leftarrow z + \Delta z$) in the following order: sequence attention, sequence transition, sequence-to-pair projection, Triangle Multiplication (outgoing), Triangle Multiplication (incoming), Triangle Attention (starting node), Triangle Attention (ending node), pair transition.

1.2 Triangle Multiplication

For the outgoing variant, with channel dimension $C = 128$, the update is:

```
z_ln = LayerNorm_in(z)
a     = mask ⊙ sigmoid(W_ag · z_ln) ⊙ (W_ap · z_ln)           (per token, 128 → 128)
b     = mask ⊙ sigmoid(W_bg · z_ln) ⊙ (W_bp · z_ln)
x[i][j][c] = Σ_k a[i][k][c] · b[j][k][c]                     (einsum "ikc,jkc->ijc")
g     = sigmoid(W_g · z_ln)
Δz    = g ⊙ (W_z · LayerNorm_out(x))
```

All operations except the einsum are token-local: each token is processed using only its own 128 values and the weights. The einsum is the only cross-token operation. For each channel c it is an $L \times L$ matrix product, $x_c = a_c \cdot b_c^T$; the 128 channels are independent (batched matrix multiplication). The incoming variant computes $x[i][j][c] = \sum_k a[k][i][c] \cdot b[k][j][c]$, i.e. $a_c^T \cdot b_c$.

Per output token, the einsum costs $2 \cdot L \cdot C$ operations (256,000 at $L = 1,000$), and the six 128×128 linear layers cost $6 \cdot 2 \cdot C^2 = 196,608$ operations. The einsum therefore dominates TriMul arithmetic as L grows.

Operation types in one Triangle Multiplication, per output token:

Operation	Type	Operations per token	Data dependence
LayerNorm_in, LayerNorm_out	normalization (reduction over 128 channels)	$O(C)$	token-local
6 linear layers (128×128)	matrix-vector	$6 \cdot 2C^2 = 196,608$	token-local, shared weights
sigmoid, gating, mask	element-wise	$O(C)$	token-local
einsum	batched matrix multiplication	$2 \cdot L \cdot C$	reads row i of a and row j of b

1.3 Triangle Attention and scaling

Triangle Attention applies attention along rows (or columns) of z . Its attention logits have shape $L \times L \times L$ per head, so they grow cubically; all other pair-representation tensors grow quadratically. For the starting-node variant, with $H = 4$ heads of dimension 32:

```

x = LayerNorm(z)
q, k, v = W_q x, W_k x, W_v x                (per token, split into H heads)
β[h][j][k] = (W_β x)[j][k][h]                (pair bias)
s[i][h][j][k] = q[i][j][h] · k[i][k][h] / √32 + β[h][j][k]
o[i][j][h] = Σ_k softmax_k(s[i][h][j][·]) · v[i][k][h]

```

The ending-node variant applies the same computation to the transpose of z .

Tensor	Shape	Growth
Sequence representation	$L \times 1024$	$O(L)$
Pair representation and TriMul intermediates	$L \times L \times 128$	$O(L^2)$
Triangle Attention logits (per head)	$L \times L \times L$	$O(L^3)$

The LightNobel paper reports that at $L = 2,034$ the activations are already $24.15 \times$ larger than the weights and require 144 GB, exceeding a single GPU. Its execution-time profile on

an H100 without chunking (Fig. 3) shows the shift in bottleneck:

Share of total time	Short protein (R0271)	Long protein (T1269)
Protein Folding Block	83.8%	94.5%
Pair-representation dataflow	69.4%	91.9%
Triangle Multiplication	36.1%	14.5%
Triangle Attention	29.0%	75.9%

2. LightNobel

2.1 Overview

LightNobel targets the Protein Folding Block only; ESM-2 and the structure module are not mapped to the accelerator. The design rests on four observations:

1. For long sequences, execution time is concentrated in the pair-representation dataflow, especially Triangle Attention.
2. Memory is dominated by activations, not weights.
3. Values vary little across the channels of one token but strongly across tokens, and outliers concentrate in specific tokens (correlated with distogram patterns). Token-wise quantization therefore fits better than channel-wise quantization.
4. Activations at different positions in the block have different statistics (e.g. mean absolute value about 82 before LayerNorm on the residual path vs. about 4 elsewhere).

The contributions are: Token-wise Adaptive Activation Quantization (AAQ) in software; a Reconfigurable Matrix Processing Unit (RMPU), a Versatile Vector Processing Unit (VVPU) and a Token Aligner in hardware; and a token-wise dataflow that avoids storing large intermediates, including the L^3 attention logits.

2.2 Token-wise Adaptive Activation Quantization (AAQ)

Each token t is quantized symmetrically with its own scale:

$$\begin{aligned} s_t &= \max_c |x_t[c]| / (2^{(b-1)} - 1) & (b = 4 \text{ or } 8) \\ q_t[c] &= \text{round}(x_t[c] / s_t) \\ \hat{x}_t[c] &= s_t \cdot q_t[c] \end{aligned}$$

Outliers are the k values of largest magnitude in each token. They are selected at run time with a top- k operation and stored in INT16 at original precision; the remaining values (inliers) are quantized. Activations are divided into three groups by their position in the block:

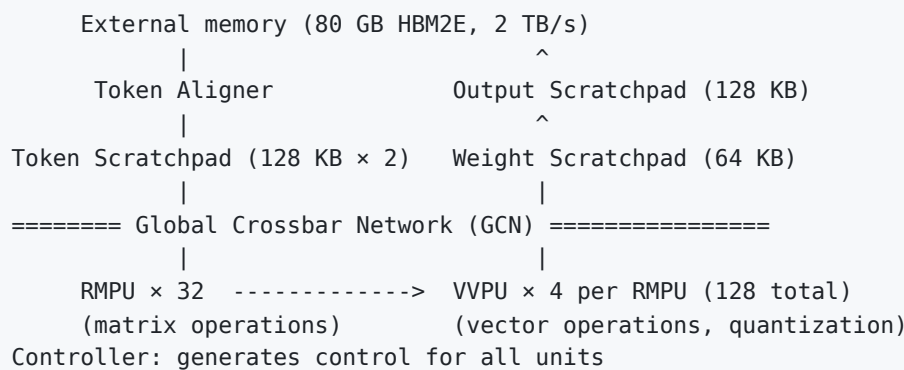
Group	Activations	Mean abs. value	Avg. outliers per token	Format
A	before LayerNorm, on the residual path	82.14	2.31	INT8 inliers + 4 INT16 outliers
B	after LayerNorm, before linear layers	4.05	1.69	INT4 inliers + 4 INT16 outliers
C	all other activations (e.g. linear outputs)	3.85	0.64	INT4, no outlier handling

Additional design points:

- Weights are not quantized; they are stored in INT16. The baseline (ESMFold) uses FP16 for weights and activations.
- Symmetric quantization is used because, with outlier handling, its RMSE is only 9.76% higher than asymmetric quantization (27.35% without outlier handling), and it is cheaper in hardware.
- The per-group settings were chosen by design-space exploration over precision and outlier count, evaluated by TM-score and data size. Rejected alternatives include: INT4 for group A (needs 32 or more outliers to keep accuracy), fewer than 4 outliers for group A (TM-score drops), INT8 for group B (accurate but larger), and channel- or tensor-wise granularity.
- Storage format per token: inliers, then outliers, then the scale, then outlier indices. Tokens are packed into blocks sized to the memory-channel bandwidth; compressed token length differs by group.

In the Triangle Multiplication, z belongs to group A, $z_{\ln}$ to group B, and a , b to group C (INT4).

2.3 Architecture and execution model



The units are time-shared rather than wired as a fixed pipeline for one layer:

- The RMPU executes all matrix-type operations: linear layers, the TriMul einsum, $Q \cdot K^T$ and attention $\cdot V$.
- The VVPU executes element-wise and vector operations: LayerNorm, sigmoid, gating, residual addition, softmax, and run-time quantization.
- The GCN routes data between units in any order (e.g. VVPU $\rightarrow$ RMPU $\rightarrow$ VVPU).

A model layer executes as a sequence of passes. In each pass, the Token Aligner reads a block of packed tokens from HBM, decodes it and writes one token per scratchpad row. The Token Scratchpad is double-buffered, so the next block loads while the current one is processed. Weights for the pass are held stationary in the Weight Scratchpad. The VVPU performs any preprocessing (e.g. LayerNorm), the RMPU performs matrix operations on up

to 20 tokens at a time, and results stream directly into the VVPU for post-processing and run-time quantization. Quantized tokens are written back through the Output Scratchpad. Latency is determined by the slowest pipeline stage (RMPU, VVPU, or memory).

Weight-stationary dataflow is chosen because a 128×128 INT16 weight matrix is only 32 KB but is reused by up to L^2 tokens.

2.4 RMPU: bit-decomposed multi-precision matrix unit

Mixed INT4/INT8/INT16 operands would leave dedicated per-precision multipliers idle. The RMPU instead uses one kind of small multiplier (following Bit Fusion) and composes larger products from 4-bit chunks:

$$A = \sum_i a_i \cdot 2^{(4i)}, \quad B = \sum_j b_j \cdot 2^{(4j)}$$

$$A \cdot B = \sum_i \sum_j (a_i \cdot b_j) \cdot 2^{(4(i+j))}$$

The shifts are implemented without multipliers. An $\text{INT4} \times \text{INT16}$ product needs 4 chunk products, $\text{INT8} \times \text{INT16}$ needs 8, and $\text{INT16} \times \text{INT16}$ needs 16.

In two's complement, only the most significant chunk is signed; the lower chunks are unsigned. The Reconfigurable Data Aligner (RDA) sign-extends the top chunk and zero-extends the others to 5 bits. Every chunk then fits one 5-bit signed format (-16 to 15), so the multipliers take 5-bit signed inputs and a single multiplier type handles all chunks.

Hierarchy:

Level	Composition	Multipliers
PE	16 multipliers + shifters + 16-to-1 adder tree	16
PE Lane	8 PEs	128
PE Cluster	20 PE Lanes + Dynamic Accumulation Logic (DAL)	2,560
RMPU Engine	4 PE Clusters (80 lanes, 640 PEs)	10,240

A 128-element dot product of a group-C token (128 INT4) with INT16 weights needs $128 \times 4 = 512$ multipliers (4 lanes); a group-B token ($124 \text{ INT4} + 4 \text{ INT16}$) needs $124 \times 4 + 4 \times 16 = 560$ (5 lanes). A cluster of 20 lanes, the least common multiple of 4 and 5, is fully utilized in both cases (5 or 4 tokens at a time).

The DAL handles scaling. Inlier partial sums must be multiplied by the token scale; outlier partial sums (INT16 originals) must not be. For 4-lane tokens, the lane sums are added and scaled once. For 5-lane tokens, the four inlier lanes are added and scaled, and the outlier lane is added afterwards. An arbiter and reconfigurable adder trees switch between 4-to-1 and 5-to-1 reduction. Results can be tapped at several levels of the adder tree: 2-PE sums

for head dimension 32 in attention, 4- or 5-lane sums for quantized linear layers, and 8- or 16-lane sums for unquantized operands.

2.5 VVPU: vector unit and run-time quantization

Each VVPU contains 128 SIMD lanes (one per channel of a token, each with a 16-bit ALU, a local scratchpad and a two-level exponent lookup table), a Scalar Support Unit (SSU) for reductions and format handling, and a Local Crossbar Network (LCN) for data exchange between lanes.

Top-k selection uses a bitonic sorting network. It is a fixed network of compare-and-swap elements with data-independent control; for 128 values it needs $7 \cdot 8 / 2 = 28$ stages of 64 comparators, and can terminate early for top-k. Original indices are tracked to record outlier positions, and $k = 1$ yields the maximum used for softmax and scale computation.

Run-time quantization proceeds in four steps: (1) top-k selection of outliers and the scale, (2) scaling and rounding of inliers, (3) reordering into the storage format through the LCN, and (4) final alignment by the SSU. Full-precision values are never written to external memory.

The paper's design-space exploration (Fig. 12) shows that latency saturates at 4 VVPUs per RMPU and at 32 RMPUs. Beyond these points the added units are not on the critical path, or the memory system cannot supply more data.

2.6 Token-wise multi-head attention

To avoid storing the L^3 logits, LightNobel computes attention token-wise in a manner similar to FlashAttention. For each query, scores are processed in blocks with running statistics:

```
m_new = max(m, max(s))
ℓ      = ℓ · exp(m - m_new) + Σ exp(s - m_new)
o      = o · exp(m - m_new) + Σ exp(s - m_new) · V
m      = m_new
output = o / ℓ
```

On the hardware, the RMPU computes $Q \cdot K^T$ per head in parallel, the VVPU dequantizes and accumulates, and softmax for one block overlaps with $Q \cdot K^T$ for the next; V is then applied and the result written back. The small hidden dimension (128) allows many tokens to reside on chip simultaneously, which makes the token-wise approach efficient for PPMs.

2.7 Mapping of Triangle Multiplication (*our interpretation*)

The paper does not list per-operation schedules. Based on operation dependencies and the described capabilities, TriMul executes in three passes:

Pass	Scope	Operations
1	token-local, over (i, k)	read z → VVPU LayerNorm → RMPU four linear layers → VVPU sigmoid, gating, mask, INT4 quantization → write a , b
2	cross-token	read a , b → RMPU einsum → VVPU dequantization and accumulation
3	token-local, over (i, j)	VVPU LayerNorm_out → RMPU w_z , w_g → VVPU sigmoid, gating, residual, INT8 + outlier quantization → write z

a and b must be written to memory between passes 1 and 2 because the einsum reads entire rows across tokens.

2.8 Evaluation methodology and results

The paper evaluates the design with the following tools:

- A Python cycle-accurate simulator, cross-validated against RTL simulation (mean error 3.30%, all within 5%).
- SystemVerilog RTL synthesized with Synopsys Design Compiler at 28 nm and 1 GHz.
- CACTI 7.0 and a memory compiler for SRAM, scaled to 28 nm.
- Ramulator for 80 GB HBM2E at 2 TB/s.

The workload is ESMFold (ESM-2 3B) on CAMEO, CASP14, CASP15 and CASP16, compared with NVIDIA A100 and H100.

Metric	Result
Accuracy	TM-score change < 0.001
Speedup	up to 8.44× (A100), 8.41× (H100); 3.85–8.44× and 3.67–8.41× vs. GPUs with chunking
Power efficiency	up to 37.29× (A100), 43.35× (H100)
Peak memory	up to 120.05× lower than unchunked GPU; 1.26–5.05× lower than chunked GPU
Maximum length	9,945 residues within 80 GB (1.45× the longest CASP16 protein, 6,879)
Area / power	178.80 mm ² , 67.8 W

The area and power breakdown (Table 2) is dominated by data movement. The crossbars (128 LCNs + GCN, about 125.7 mm²) take 70.28% of area and 67.95% of power, while the RMPU engines take 18.20% of area. Within one VVPU, the LCN (0.785 mm²) is about 7× larger than the 128 SIMD lanes (0.115 mm²).

Memory savings come from two separate mechanisms:

- **Quantization.** With the dataflow unchanged, total memory drops from 121.39 GB to 73.50 GB (1.65×; T1169, 3,364 residues, Table 1).

- **Not materializing intermediates.** In particular, removing the L^3 attention logits accounts for most of the 120× peak reduction relative to an unchunked GPU.

The pair representation still grows as L^2 .

2.9 Related approaches

Approach	Examples	Effect	Limitation
GPU system optimization	FastFold, ScaleFold	faster training and inference	memory still grows as L^2
Chunking	AlphaFold/ESMFold chunk option, AutoChunk	lower peak memory	slower; repeated reads
Model quantization	MEFold (weights), PTQ4Protein (per-tensor INT8)	smaller data	weight-only quantization has limited effect on activations; accuracy loss at low precision
LLM quantization	SmoothQuant, LLM.int8(), AWQ	outlier separation	single inlier precision; activation quantization is often slower on GPUs
Fused GPU kernels	OpenFold3 fused TriMul	intermediates not materialized	hand-written kernels, GPU-specific
Dedicated accelerator	LightNobel	quantization, specialized units and token-wise dataflow	simulated, not fabricated

3. FPGA Implementation of the Triangle-Multiplication Einsum

3.1 Target and kernel specification

Item	Value
Board	AMD Alveo U55C, 16 GB HBM2 (32 pseudo-channels, ~14 GB/s each, ~460 GB/s total)
Toolchain	Vitis HLS (C++ → RTL), Vitis v++ linking, XRT host
Kernel clock	300 MHz target (3.33 ns); HLS estimated critical path 2.431 ns
Operation	$z[i][j][c] = \sum_k a[i][k][c] \cdot b[j][k][c]$, $C = 128$, FP32
Layout	<code>[row][col][channel]</code> , index <code>(row · L + col) · C + c</code> (same as PyTorch)
Tile	$T_I = T_J = 8$

With this layout the 128 channels of one token are contiguous: 512 bytes, or exactly eight 512-bit words.

U55C memory hierarchy:

Level	Capacity	Access latency
Flip-flops	about 330 KB	1 cycle
BRAM	$4,032 \times 18 \text{ Kb} \approx 9 \text{ MB}$	1-2 cycles
URAM	$960 \times 288 \text{ Kb} \approx 35 \text{ MB}$	1-2 cycles
HBM2	16 GB ($32 \times 512 \text{ MB}$)	tens to hundreds of cycles

One $L \times L \times 128$ FP32 tensor at $L = 1,000$ occupies 512 MB, compared with about 43 MB on chip. The einsum therefore has to stream tiles from HBM.

Correctness was checked in two ways:

- **C simulation.** The testbench uses random activations with 1% of tokens scaled by $30\times$ and compares against a double-precision CPU reference. v0-v3 all give $\text{rel_l2} = 1.141 \times 10^{-7}$ (identical results).
- **Equivalence with ESMFold.** Using the HuggingFace ESMFold implementation, `einsum("ikc,jkc->ijc", a, b)` equals the module's internal computation with maximum absolute difference 0.

3.2 v0: baseline

```
for i, for j, for c:
    acc = 0
    for k: acc += a[(i*L + k)*C + c] * b[(j*L + k)*C + c]
    z[(i*L + j)*C + c] = acc
```

Every multiply-accumulate reads two operands from HBM, so there is no on-chip reuse. The innermost loop advances k , which moves the address by C elements per iteration. The access pattern is strided, HLS cannot infer bursts, and each access pays the full memory latency.

3.3 v1: loop tiling with on-chip buffers

```
for (i0 = 0; i0 < L; i0 += TI) for (j0 = 0; j0 < L; j0 += TJ):
    acc[TI][TJ][C] = 0 // BRAM, 32 KB
    for k:
        load_a: la[ii][c] ← a[((i0+ii)*L + k)*C + c] // TI tokens, 4 KB
        load_b: lb[jj][c] ← b[((j0+jj)*L + k)*C + c] // TJ tokens, 4 KB
        mac: acc[ii][jj][c] += la[ii][c] * lb[jj][c] // ii, jj, c
    store acc → z
```

- **Tiling and reuse.** The output space is partitioned into $T_I \times T_J$ tiles whose accumulators stay on chip (output-stationary). For each k , T_I tokens of `a` and T_J tokens of `b` are loaded once and reused inside the `mac` loop: each `la` element serves T_J outputs and each `lb` element serves T_I outputs. Off-chip reads fall by 8× for 8×8 tiles.
- **Burst inference.** The innermost load loop iterates over c at consecutive addresses, which allows HLS to infer one AXI burst of 128 beats per token; the latency is then paid once per token instead of once per element.
- **Pipelining.** All loops are pipelined with $II = 1$. A dependence pragma (`inter false`) on `acc` states that the same accumulator is updated only once every $T_I \cdot T_J \cdot C$ iterations, so floating-point adder latency does not limit the initiation interval.
- **Limitation.** The 32-bit ports and a single MAC per cycle remain. Each k iteration costs about $1,024 + 1,024 + 8,192$ cycles, dominated by computation (estimate).

3.4 v2: wide memory interface and vectorization

v2 widens the interface, the on-chip buffers and the datapath to 16 lanes at once:

- **Interface.** Pointer types change from `float*` to a 512-bit struct of 16 floats (`f16`), so HLS generates 512-bit AXI master ports. `a`, `b` and `z` use separate bundles (`gmem0/1/2`), giving three independent ports mapped to different HBM pseudo-channels. Per-port bandwidth rises from 1.2 GB/s to 19.2 GB/s at 300 MHz. One token

becomes a burst of 8 beats, and multiple outstanding requests overlap the latency of consecutive tokens.

- **Buffers.** `la` , `lb` and `acc` are declared `[rows][16]` and completely partitioned in the lane dimension, so 16 values can be read and written per cycle.
- **Datapath.** The lane loop inside the pipelined `mac` loop is fully unrolled into 16 parallel floating-point MAC units (peak $16 \times 2 \times 300 \text{ MHz} = 9.6 \text{ GFLOP/s}$).

HLS report ($L = 256$, $T_I = T_J = 8$):

Module	Latency (cycles)	Decomposition
init	514	512 + 2
load_a / load_b (parallel)	141	64 words + 77
mac	526	$8 \cdot 8 \cdot 8 = 512$ + pipeline depth 14
loop_k iteration	671	141 + 526 + 4
store	586	512 + 74
tile	172,880	
total	177,029,185 (0.590 s)	1,024 tiles

3.5 v3: double buffering

In v2, loading and computation alternate: 141 of every 671 cycles leave the MAC units idle. v3 overlaps them:

- **Buffers.** Two buffer sets (`la0/lb0` , `la1/lb1`).
- **Modules.** Loading and computation are separate functions (`load_ab` , `mac`) with `INLINE off` , so each becomes an independent hardware module.
- **Schedule.** Before the `k` loop, `k = 0` is loaded into set 0. For even `k`, `load_ab(k+1 → set 1)` runs concurrently with `mac(set 0)` ; for odd `k` the roles swap. Because the two calls access disjoint arrays, HLS schedules them in parallel.
- **Why two sets.** A single buffer would create a write-after-read hazard: the next load would overwrite data the current `mac` is still reading.
- **Cost.** BRAM for `la / lb` doubles. Off-chip traffic, port width and MAC count are unchanged.

```
v2:  [load k=0][ mac k=0 ][load k=1][ mac k=1 ][load k=2][ mac k=2 ] ...
v3:  [load k=0][ mac k=0 ][ mac k=1 ][ mac k=2 ][ mac k=3 ] ...
      [load k=1] [load k=2] [load k=3]
```

The iteration time changes from a sum to a maximum:

$v2: T_{\text{iter}} = T_{\text{load}} + T_{\text{mac}} + 4 = 141 + 526 + 4 = 671 \text{ cycles}$
 $v3: T_{\text{iter}} = \max(T_{\text{load}}, T_{\text{mac}}) + 4 = 530 \text{ cycles (model prediction)}$
 per tile: $v2 = L \cdot 671$, $v3 = 141 + L \cdot 530 \rightarrow \text{speedup} \rightarrow 671 / 530 \approx 1.27$ for large L

The gain is bounded by the load share (21%), following Amdahl's law. The v3 figure is a model prediction pending HLS synthesis.

3.6 Summary of v0-v3

Version	Technique	Off-chip reads (elements)	MACs per cycle	Arithmetic intensity (FLOP/byte)	Status
v0	baseline	$2 L^3 C$	1	0.25	C-sim verified
v1	loop tiling, on-chip buffers, burst access	$L^3 C (1/T_I + 1/T_J)$	1	2	C-sim verified
v2	512-bit AXI, array partitioning, 16-lane SIMD	same as v1	16	2	C-sim verified, HLS report
v3	double buffering	same as v1	16	2	C-sim verified, HLS pending

4. Computation Methods

4.1 Execution-time model

```
no overlap:  T ≈ T_compute + T_memory
overlap:     T ≈ max(T_compute, T_memory)
T_compute = operations / (operations per cycle × f_clk)
T_memory   = bytes / bandwidth + latency × number of non-overlapped requests
```

Each optimization acts on one term:

Technique	Term affected	Version
Tiling	bytes ↓	v1
Wide interface, bursts	bandwidth ↑, non-overlapped requests ↓	v1, v2
Parallel MAC units	operations per cycle ↑	v2
Double buffering	sum → max	v3
Multiple HBM channels	bandwidth ↑	configuration study

4.2 Off-chip traffic

```
v0:  R_0 = 2 · L3 · C
v1:  R_1 = (L/T_I)(L/T_J) · L · (T_I + T_J) · C = L3 · C · (1/T_I + 1/T_J)
R_0 / R_1 = 2 / (1/T_I + 1/T_J) = 8 (8×8), 16 (16×16), 32 (32×32)
```

4.3 Arithmetic intensity and roofline

```
AI = FLOPs / bytes = 2 L3 C / (4 · R)
v0: AI = 2 L3 C / (4 · 2 L3 C) = 0.25 FLOP/byte
v1-v3: AI = T_I · T_J / (2 (T_I + T_J)) = 2 (8×8), 4 (16×16), 8 (32×32)
attainable performance = min(peak compute, AI × bandwidth)
```

In v2, the 16 MAC units consume at most 9.6 GFLOP/s. At AI = 2 FLOP/byte this requires 4.8 GB/s, well below the 38.4 GB/s supplied by the two 512-bit input ports, so the tiled kernel is compute-bound. This agrees with the HLS breakdown, in which `mac` takes 526 of 671 cycles per iteration. v0 is memory-bound: each MAC waits for two non-burst accesses.

4.4 Cycle model and calibration

```
cycles_total = (L/T_I)(L/T_J) · cycles_tile
cycles_tile  = t_init + L · t_iter + t_store + ε
t_load       = max(T_I, T_J) · C/16 + α      (α: memory latency and loop start-up)
t_mac        = T_I · T_J · C/16 + δ          (δ: pipeline depth)
t_iter       = t_load + t_mac + 4            (v2)
t_iter       = max(t_load, t_mac) + 4        (v3)
t_store      = T_I · T_J · C/16 + σ
```

The constants were calibrated from the v2 HLS report:

Report value	Structural term	Calibrated constant
load = 141	$8 \cdot 8 = 64$	$\alpha = 77$
mac = 526	$8 \cdot 8 \cdot 8 = 512$	$\delta = 14$
store = 586	512	$\sigma = 74$

The calibrated model predicts 177,029,120 cycles for v2, against 177,029,185 in the report ($< 0.01\%$ error). Since calibration and comparison use the same report, this checks consistency rather than predictive accuracy. Validation uses configurations not seen during calibration: tile sizes 16×16 and 32×32 (`make hls-sweep`) and versions v3 and v4 (`make hls-all`), with a target error below 10%.

The model has three limitations:

- HLS latencies are estimates.
- Contention between ports sharing an HBM channel is not modeled.
- Resource figures (DSP, BRAM) are indicative only.

4.5 On-chip memory

```
acc = T_I · T_J · C · 4 B      (8×8: 32 KB; 16×16: 128 KB; 32×32: 512 KB)
la  = T_I · C · 4 B, lb = T_J · C · 4 B  (8 tokens: 4 KB each; doubled in v3)
```

Larger tiles increase arithmetic intensity but consume BRAM quadratically. The design-space exploration selects tile size at the knee of the cycles-versus-BRAM Pareto front.

4.6 Capacity model

Memory requirement scales with L^2 , so the maximum sequence length scales with the square root of the available memory:

```
memory = L2 · (bytes per pair position)
L_max  = sqrt(capacity / bytes per pair position)
```

For the 16 GB U55C, with an FP16 baseline (the precision used by ESMFold and by the LightNobel comparison):

Configuration	Bytes per L^2 position	L_{max}	Relative
Unfused (≈ 7 live $L \times L \times 128$ tensors, FP16)	$7 \times 256 = 1,792$	$\approx 3,096$	1.00×
Fused producer/consumer, FP16 z , INT8 a , b (design estimate)	$2 \times 256 + 2 \times 132 = 776$	$\approx 4,705$	1.52×
Same with INT4 a , b (design estimate)	$2 \times 256 + 2 \times 68 = 648$	$\approx 5,148$	1.66×

INT8 tokens are 128 B plus a 4-byte scale; INT4 tokens are 64 B plus a 4-byte scale. The fused configurations are analytical estimates and have not been implemented. Because L grows only with the square root of the memory reduction, a 2.3× reduction in bytes per position yields about 1.5× in L .